

Parallel Project Report:(Distributed SpMV)

Andrea Dalla Villa (ID: 242637)

Email: andrea.dallavilla@studenti.unitn.it

Course: Introduction to Parallel Computing (2025–2026)

Abstract—Sparse matrix vector multiplication (SpMV) is one of the most computational kernels used in different fields and analyzed many times to obtain the best possible performance solution. In the previous paper I analyzed how this kernel works in a shared memory system using a SIMD model. Now I am going to observe what happens in a distributed memory system using a SPMD model thanks to the MPI library used for message-passing among processes into our HPC cluster. I will explain the round trip time (RTT) and the execution time for SpMV and calculate the speedup and efficiency per rank using a 1D partition.

Index Terms—SpMV, MPI, processes, 1D

I. INTRODUCTION

This section discusses the reasons why some scientific papers are careful about the sparse matrix vector multiplication; it is sufficient to observe how redundant the computation is and how much time we are wasting while doing it. CPUs do these kinds of operations every time, and often they are more complex than SpMV; furthermore, whether we choose a sparse matrix instead of a dense matrix we risk to compute some useless operations with a large number of zero values. The main focus of this work is how a good message-passing with processes and a good work balance can increase drastically performance using a standard partition like 1D is there in many matrix libraries. To begin with, I would like to specify many considerations, for instance:

- **Sparsity**, since values are not allocated contiguously, a high level of it does not exploit the spatial locality if we want to consider only non zero values.
- **The structure of the matrix**; the results depend on whether diagonal, symmetric, or irregular sparse matrices are used; another important factor is the number of non-zero values, furthermore, these parameters can be the reason why you risk the memory-bound caused by the load imbalance among processes especially in a 1D partitioning [3].
- **Architecture and network topology used**, when you increase the number of computations, you should use an appropriate architecture to measure better performances. For instance, performance can have different results for an architecture with fewer cores or if it is a NUMA or an UMA architecture. Even the network topology (e.g. ring, dragonfly or fat-tree) plays a crucial role in terms of latency, bandwidth and diameter.

II. STATE OF THE ART

Improving SpMV performance is highly relevant, given that we live in a multidimensional world where everything is represented by 2D/3D matrices and a great SpMV algorithm can change performance for many engineering fields like machine learning with deep neural networks [4], where SpMV plays an important role. Another field is when we have spoken about images or videos [5] and always new approaches for this computational kernel are implemented to maximize performance, like CNN [6] tanks supercomputers like Leonardo.

III. CONTRIBUTION AND METHODOLOGY

A. Local SpMV algorithm

I download four matrices by Suitsparse [1] for the strong scaling and I create a synthetic matrix for the weak scaling. I have used a quick sort algorithm for sorting the mtx files by row and after I send the vector to other ranks, the number of non zeros that each rank will receive and pieces of COO divided by row follow the 1D partitioning. I have converted them to a local CSR format and compute the local SpMV multiplication. This is the algorithm used in the first deliverable that I have taken from [2] and implemented in C.

The CSR Format Sparse Matrix-Vector Multiplication Algorithm

Require: N : the total number of rows in matrix $\mathcal{A}$
 val : nonzero values in matrix $\mathcal{A}$
 col : column indices of values in matrix $\mathcal{A}$
 row : pointers to row starts in matrix $\mathcal{A}$

Input : X : the vector need to calculate

Output : Y : the vector after calculate

```
1 for  $i \leftarrow 0$  to  $N - 1$  do
2   for  $j \leftarrow row_i$  to  $row_{i+1}-1$  do
3      $Y_i \leftarrow Y_i + val_j * X_{col_j}$ 
4   end
5 end
```

Fig. 1. Pseudo code of the algorithm

The last but not the least, rank 0 receives all pieces of result from the other ranks and merges them.

B. Message passing interface (MPI)

I used several Root Collectives to share data between processes for instance:

- **int MPI_Bcast(...);** for the vector, number of ROW, COLUMN and NON-ZEROS.
- **int MPI_Scatter(...);** to share the ranks' non-zeros number.
- **MPI_Reduce(...);** to receive the maximum local SpMV execution time among ranks.
- **V-Collectives,** useful when the buffer-size is different among processes, like:
- **MPI_Scatterv(...);** to share the pieces of COO (Instead of scattering three separate arrays (row, col, val), I define an MPI derived datatype for COO and scatter the non-zeros with a single MPI_Scatterv, reducing the number of MPI calls).
- **MPI_Gatherv(...);** to collect the partial results from the processes and prepare the final result.

IV. EXPERIMENTS AND SYSTEM DESCRIPTION

A. Platform & matrices

I have done my project on the HPC cluster, an Intel x86_64 architecture with 2 head nodes and 142 CPU for a total of 7674 cores and 10 GPU for a total of 48128 CUDA cores inside the computing nodes. Any node has a NUMA behavior with 4 blocks and each one has 18 CPUs I have run my program on a head node with a pbs where I could choose the number of cores, processes that I needed and the number of nodes. I fix the total number of cores to 64 and allocate 1 MB of memory per process. For each experiment, I modify only the number of nodes and the number of processes.

OS: x86_64 GNU LINUX

compiler: gcc version 9.1.0.

Dataset: For the strong scaling I downloaded different structures with different sparsity level on "Suitsparse" website [1] e.g. a diagonal matrix like "cage15", pretty dense matrices like "F1" and "ship_001" and a graph like "Spielman_k500".

matrix	rows number	columns number	non zeros
Spielman_k500	41.792.002	41.792.002	83.709.503
cage15	5.154.859	5.154.859	99.199.551
F1	343.791	343.791	13.590.452
ship_001	34.920	34.920	2.339.575

For the weak scaling I create this matrix that changes its dimension for each process, starting from 10.000 rows and cols and 500.000 non-zeros, to 2.560.000 rows and cols and 256.000.000 non-zeros.

B. Methodology

I report:

- **computation execution time especially for weak scaling,** the execution time for SpMV registered by the slowest rank.
- **RTT,** it includes both communication time (MPI_Collectives for data distribution and result

gathering) and computation time (local SpMV execution measured on the slowest rank);

- **Speedup,** the RTT with one process divided by the RTT obtained with a different number of processes;
- **Efficiency per rank,** It is the only SpMV speedup(calculated using the computation execution time) divided by the number of processes;
- **FLOPs,** The total number of computation in a SpMV, I report also the FLOPs per rank(It is an estimate);
- **NNZ min,max,avg per rank and (max/avg),** useful to analyze the load balance;

V. RESULTS AND DISCUSSION

A. Strong scaling vs Weak scaling

In this section, I compare the performance when we take real matrices (strong scaling) with when we create a matrix where the number of non-zeros and the dimension rise with the number of processes used. We can start by showing the Round Trip Time (RTT), defined as the end-to-end execution time of a distributed SpMV iteration, measured from data distribution to final result collection. This time includes the communication ex.time and the computation ex.time (SpMV execution time by the slowest rank).

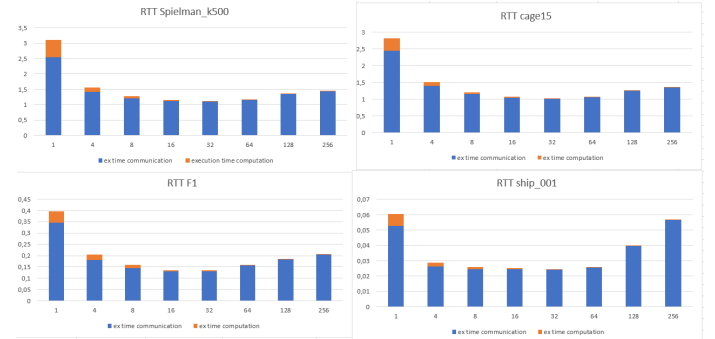


Fig. 2.

We can see that the communication uses much time in comparison with the computation, indeed, when we increase the number of processes each process has few number of non-zeros than before, but process 0 has to send and wait data to many processes. A communication part into RTT is observed even with a single process due to the overhead of MPI_Collectives call, which are invoked consistently across all configurations. Another curious fact regards the matrix ship_001 where in the previous paper good performance was obtained compared to others; In a distributed memory architecture we can increase the dimension and improve performance compared to use a shared memory architecture, each process has a local memory and no other processes overwrite it, this reduces cache misses and improves the scalability. Furthermore, a matrix categorized big in a shared memory architecture, here it is too small, we can see that when we improve the number of processes much time is wasted and each process has a few workload. A way to see how many computations a process has

is to consider FLOPs per rank, a FLOP is the product among non-zeros*2 since one computation is a sum of product. in the table below there are the avg of FLOPs per process.

processes	FLOPs Spielman_k500	FLOPs cage15	FLOPs F1	FLOPs ship_001
1	167419006	198399102	27180904	4679150
4	41854752	49599776	6795226	1169788
8	20927376	24799888	3397613	584894
16	10463688	12399944	1698807	292447
32	5231844	6199972	849403	146223
64	2615922	3099986	424702	73112
128	1307961	1549993	212351	36556
256	653980	774996	106175	18278

With small matrices, it is not convenient to increase the number of processes because processes waste time waiting for data. in the two graphs below, we observe the speedup and the efficiency(per rank); a good speedup is close to 2-3 and a good efficiency to 1-0,9.

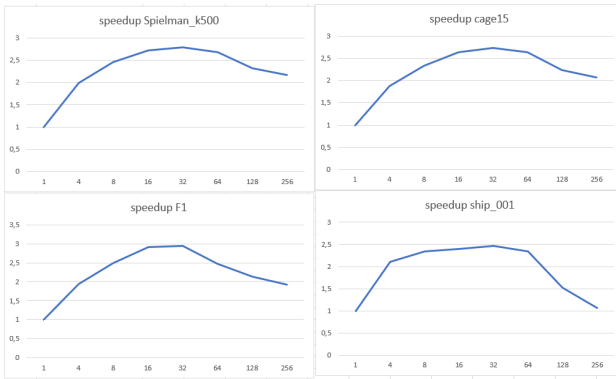


Fig. 3.

The best performance obtained is around 32 processes and then decreases drastically for small matrices. We can see that from 1 to 4 processes it doubles; speedup is influenced by the HPC cluster architecture, with communication latency, diameter, network topology, and process placement significantly affect scalability. HPC has a diameter of 2-3 hops and is mainly a dragonfly topology. From 1 to 64 processes, I have preferred to use intra-node processes since they are faster than inter-node processes that I had to use for 128 and 256 processes.

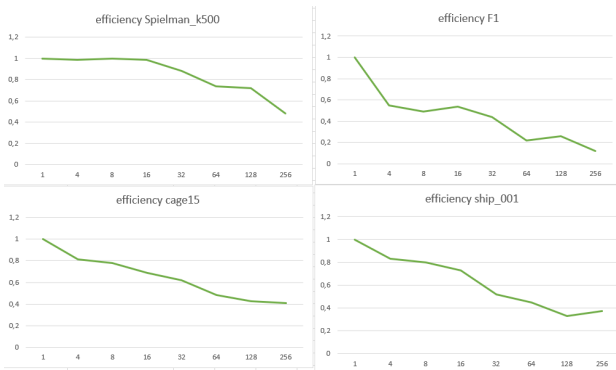


Fig. 4.

Figure 4 shows that "Spielman_k500" has a high load balance among processes; this is due to its predictable structure that distributes well the workload and fast processes do not wait much time. Later, we will analyze the load balance for each matrix also affected by the 1D partitioning. after 128 processes, we have over parallelization. "Cage15" has a linear decrease, probably since each process always computes fewer FLOPs due to its diagonal pattern, which is pretty common in strong scaling. "F1" and "ship_001", have an anomalous behavior due to the fact that they have few non-zeros in comparison with the others, and they present an irregular dense pattern that causes an imbalance data passing among processes. For weak scaling the hypothetical behavior is that each process has the same execution time and the efficiency is constantly close to 1. In this real graph things change a bit: Even if the number of non-zeros is the same for each

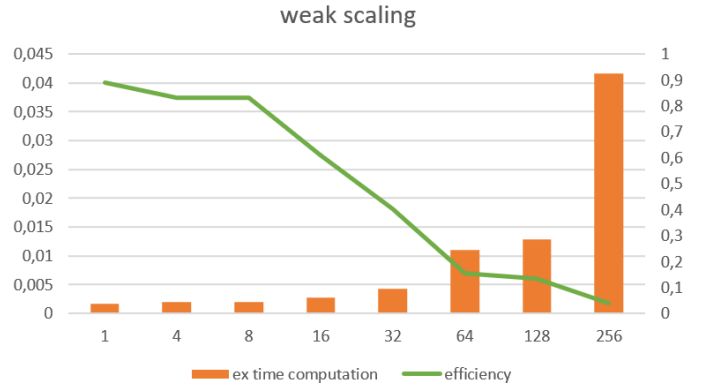


Fig. 5.

process, the position of them can change and this causes a few delay which it falls the efficiency per process; we can see it between 4 and 32 processes where the execution time is pretty constant but the efficiency changes drastically. After 64 processes the efficiency is close to 0 and execution time rises a lot. A possible reason is that we change node, and they saturate the bandwidth which it is caused by the same number of non-zeros for each process.

B. 1D partitioning and load balance

In the graphs discussed before We have seen when 1D partitioning performs better and when using another partitioning like 2D is the best solution. With constant matrices like "Spielman_k500" where you have a regular pattern and an appropriate number of non-zeros the efficiency is close to 1 for few processes; indeed 1D is better for few processes, and medium matrices as you send to each process a contiguous number of rows. Here we can see the Spielman_k500's number of non-zeros min,max,avg per rank and the max/avg that indicate when the load imbalance is perceptible.

processes	nnz min per rank	nnz max per rank	nnz avg per rank	max/avg
1	83709503	83709503	83709503	1
4	20917882	20945863	20927376	1
8	10458680	10479437	10463688	1
16	5229281	5243825	5231844	1
32	2614627	2624504	2615922	1
64	1307310	1313890	1307961	1
128	653655	657980	653981	1,01
256	326827	329645	326990	1,01

"Spielman_k500" has a fair distribution of non-zeros per rank that guarantees a perfect load balance, close to 1. when max/avg is more than 1.3; load imbalance is perceptible and when it is close to 2, the imbalance is pretty high. If we observe the "Cage15" max/avg we can see that it rises in a constant way and for few processes you have an acceptable load balance. However, with a large number of processes you have a high imbalance and the efficiency is low.

processes	nnz min per rank	nnz max per rank	nnz avg per rank	max/avg
1	99199551	99199551	99199551	1
4	19291564	29808764	24799888	1,2
8	8718004	15693066	12399944	1,27
16	3789523	8220818	6199972	1,33
32	1623351	4324980	3099986	1,4
64	704301	2264472	1549993	1,46
128	311291	1300317	774997	1,68
256	129724	667709	387498	1,72

For the previous matrices 1D partitioning works good, but for "F1" and "ship_001" a 2D partitioning is better, since when you have a huge matrix or a dense irregular matrix, It is preferable to split the row instead to share the entire row to a process. 2D partitioning presented several issues, it reduces the spatial locality and increases the overhead, it is not convenient for small computational downsides. In our case, using a 2D partitioning would have increased performance for "F1" and "ship_001", while decreasing performance for the other matrices. In the next tables we have the nnz data for "F1" and after "ship_001".

processes	nnz min per rank	nnz max per rank	nnz avg per rank	max/avg
1	13590452	13590452	13590452	1
4	1623533	4746618	3397613	1,4
8	645392	2504106	1698807	1,47
16	192637	1270219	849403	1,5
32	92858	655080	424702	1,54
64	41837	330608	212351	1,56
128	16821	167962	106175	1,58
256	4326	83976	53088	1,58

processes	nnz min per rank	nnz max per rank	nnz avg per rank	max/avg
1	2339575	2339575	2339575	1
4	428098	713481	584894	1,22
8	185677	367389	292447	1,26
16	42666	190953	146223	1,31
32	13758	116763	73112	1,6
64	6699	58981	36556	1,61
128	3291	32156	18278	1,76
256	1587	16794	9139	1,84

These parameters are the reason why the efficiency falls. When the number of processes rises and the number of non-zeros per rank decreases, you risk an interconnect-bound, the communication stop scaling and after 64 processes, where you change node, is the bound, It is easily visible on the weak scaling. I told that the 1D partitioning is not the better choice for huge and irregular matrices, the memory for each process

rises with the rows that it receives and communication rises with the ghost, when the ghost can not enter in the cache or the weak scaling stops to be constant, you are arrived to the matrix dimension limit.

VI. CONCLUSIONS

In this paper we have observed performance changes between different matrix formats and how the architecture, the partitioning and the network topology influence the results obtained, for instance the same matrices but different partitioning can change completely the results. Furthermore, we have discovered that a matrix which achieved good performance in a shared memory system, in a distributed memory system have obtained bad performance compared to the others. Distributed memory system is a great choice for tasks with a high complexity and supercomputers can improve high tech researches, although it has its limits that depend on the cluster, for instance when you send a message to another node in our HPC cluster. In conclusion, we got an idea of the potential to parallelize with message-passing using MPI opening up the possibility of trying this approach with other clusters with different network topologies.

REFERENCES

- [1] <https://sparse.tamu.edu/>
- [2] J. -L. Zhang, L. Zhuang, J. Wan, X. -H. Xu, C. -F. Jiang and Y. -J. Ren, "COSC: Combine Optimized Sparse Matrix-Vector Multiplication for CSR Format," 2011 Sixth Annual Chinagrid Conference, Dalian, China, 2011, pp. 124-129, doi: 10.1109/ChinaGrid.2011.39. keywords: Optimization;Sparse matrices;Libraries;Distribution strategy;Indexes;Pipeline processing;Arrays;COSC;SpMV;CSR;optimization,
- [3] Erik G. Boman, Karen D. Devine, and Sivasankaran Rajamanickam. 2013. Scalable matrix computations on large scale-free graphs using 2D graph partitioning. In Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC '13). Association for Computing Machinery, New York, NY, USA, Article 50, 1–12. <https://doi.org/10.1145/2503210.2503293>
- [4] M. Mahesh, S. Nalesh and S. Kala, "Hardware Acceleration of SpMV Multiplier for Deep Learning," 2021 25th International Symposium on VLSI Design and Test (VDATE), Surat, India, 2021, pp. 1-6, doi: 10.1109/VDATE53777.2021.9600988. keywords: Deep learning;Machine learning algorithms;Quantization (signal);Power demand;Very large scale integration;Performance gain;Sparse matrices;Deep learning;FPGA;Sparse matrix;Performance,
- [5] A. C. Shaji and K. Khare, "Analysis of Optimization on Sparse Matrix Vector Multiplication Model Application in Digital Signal Processing," 2024 IEEE 3rd International Conference on Electrical Power and Energy Systems (ICEPES), Bhopal, India, 2024, pp. 1-7, doi: 10.1109/ICEPES60647.2024.10653621. keywords: Energy consumption;Signal processing algorithms;Digital signal processing;Vectors;Energy efficiency;Resource management;Sparse matrices;SpMV;FPGA;HLS;OpenCL;High-Performance Computing;Parallelism;DSP,
- [6] P. Guo and C. Zhang, "Sparse Matrix Selection for CSR-Based SpMV Using Deep Learning," 2019 IEEE 5th International Conference on Computer and Communications (ICCC), Chengdu, China, 2019, pp. 2097-2101, doi: 10.1109/ICCC47050.2019.9064309. keywords: Sparse matrices;Training;Graphics processing units;Measurement;Machine learning;Feature extraction;Convolution;Convolutional Neural Network (CNN);Deep Learning;SpMV;GPU,